

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO DE JOINVILLE

GABRIELLA ARÉVALO MARQUES
HEBERT ALAN KUBIS

PROGRAMAÇÃO COM SOCKETS

Joinville - SC
2026

Sumário

Parte 1: Especificação do Protocolo (Formato RFC).....	3
1. Descrição e Estrutura do Protocolo.....	3
2. Transporte.....	3
3. Formato do Pacote.....	3
4. Características Técnicas:.....	4
Parte 2: Implementação com Sockets (Python).....	4
Servidor (server.py).....	4
Cliente (client.py).....	5
Parte 3: Avaliação de Desempenho: Overhead de Cabeçalhos.....	5
Cenário A: Payload muito pequeno (Ex: "quanto é 2+2?").....	5
Cenário B: Payload médio (Ex: "qual o sentido da vida?").....	6
Parte 4: Comparação com Protocolos Padrão.....	6
QAP vs. HTTP (Hypertext Transfer Protocol).....	6
QAP vs. MQTT (Message Queuing Telemetry Transport).....	6
Conclusão.....	7
Referências.....	7

Parte 1: Especificação do Protocolo (Formato RFC)

1. Descrição e Estrutura do Protocolo

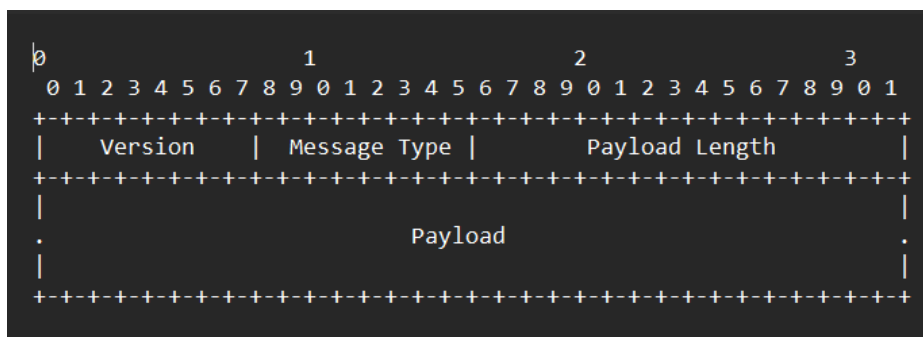
O QAP (Question and Answer Protocol) é um protocolo de camada de aplicação leve, operando sobre TCP, projetado especificamente para sistemas cliente-servidor de perguntas e respostas. O objetivo principal do QAP é minimizar a sobrecarga de cabeçalho (overhead) enquanto garante a entrega confiável das mensagens.

2. Transporte

O QAP opera sobre TCP (Transmission Control Protocol) para garantir a ordem e a confiabilidade na entrega das mensagens. A porta padrão sugerida é a 8081, embora possa ser configurada dinamicamente.

3. Formato do Pacote

Todas as mensagens QAP consistem em um cabeçalho fixo de 4 bytes seguido por um *payload* (carga útil) de tamanho variável. O formato é *Big-Endian* (Network Byte Order).



Campos do Cabeçalho:

- ★ **Version (1 Byte):** A versão atual do protocolo. Deve ser 0x01.
- ★ **Message Type (1 Byte):** Identifica o propósito da mensagem.
 - 0x01: Request (Pergunta do Cliente)
 - 0x02: Response (Resposta do Servidor)
 - 0x03: Error (Mensagem de Erro)
- ★ **Payload Length (2 Bytes):** O tamanho em bytes do campo Payload. O tamanho máximo do payload é 65.535 bytes.

Payload:

- ★ Contém a string da pergunta ou resposta, codificada em UTF-8. Não é necessário incluir caracteres nulos no final, pois o comprimento já é definido no cabeçalho.

4. Características Técnicas:

- ★ **Camada de Transporte:** TCP (garante a entrega ordenada e confiável dos pacotes).
- ★ **Serialização de Dados:** O protocolo utiliza uma abordagem binária estrita (Big-Endian, sinalizado pelo !) para o cabeçalho e codificação de texto (UTF-8) para o *payload* (corpo da mensagem).
- ★ **Cabeçalho Fixo (4 bytes):**
 - **Version (1 byte):** Define a versão do protocolo (atualmente 1).
 - **Type (1 byte):** Define a direção da mensagem (1 para Request, 2 para Response).
 - **Length (2 bytes):** Define o tamanho exato do payload. Como utiliza um inteiro curto sem sinal (unsigned short), o tamanho máximo de uma mensagem que o protocolo suporta é de 65.535 bytes (cerca de 65 KB).

Parte 2: Implementação com Sockets (Python)

Servidor (server.py)

```
server.py X cliente.py
server.py > ...
1  import socket
2  import struct
3
4  HOST = '172.20.10.2'
5  PORT = 8081
6
7  # Base de conhecimento simples
8  KNOWLEDGE_BASE = {
9      "qual e a capital do brasil?": "Brasilia",
10     "quanto e 2+2?": "4",
11     "quem e a senhora ella?": "Uma usuária muito dedicada ao seu projeto de redes."
12 }
13
14 def handle_client(conn, addr):
15     print(f"Conectado a {addr}")
16     try:
17         while True:
18             # 1. Lê os 4 bytes do cabeçalho
19             header = conn.recv(4)
20             if not header:
21                 break
22
23             # Desempacota o cabeçalho: B (unsigned char, 1 byte), B (1 byte), H (unsigned short, 2 bytes)
24             version, msg_type, payload_length = struct.unpack('!BBH', header)
25
26             if msg_type == 1: # Request
27                 # 2. Lê o payload com base no tamanho informado
28                 payload = conn.recv(payload_length).decode('utf-8')
29                 print(f"Pergunta recebida: {payload}")
30
31                 # 3. Processa a resposta
32                 pergunta_formatada = payload.lower().strip()
33                 resposta = KNOWLEDGE_BASE.get(pergunta_formatada, "Desculpe, não sei a resposta.")
34
35                 # 4. Empacota e envia a resposta
36                 payload_bytes = resposta.encode('utf-8')
37                 # Version 1, Type 2 (Response), Length
38                 response_header = struct.pack('!BBH', 1, 2, len(payload_bytes))
```

Cliente (cliente.py)

```
server.py  cliente.py X
cliente.py > ...
1  import socket
2  import struct
3
4  HOST = '172.20.10.2'
5  PORT = 8081
6
7  with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
8      s.connect((HOST, PORT))
9      print("Conectado ao servidor QAP. Digite 'sair' para encerrar.")
10
11     while True:
12         pergunta = input("Sua Pergunta: ")
13         if pergunta.lower() == 'sair':
14             break
15
16         # 1. Prepara e envia a requisição
17         payload_bytes = pergunta.encode('utf-8')
18         # Version 1, Type 1 (Request), Length
19         header = struct.pack('!BBH', 1, 1, len(payload_bytes))
20         s.sendall(header + payload_bytes)
21
22         # 2. Recebe a resposta
23         res_header = s.recv(4)
24         if not res_header:
25             break
26
27         version, msg_type, payload_length = struct.unpack('!BBH', res_header)
28
29         if msg_type == 2: # Response
30             resposta = s.recv(payload_length).decode('utf-8')
31             print(f"Resposta do Servidor: {resposta}\n")
```

Parte 3: Avaliação de Desempenho: Overhead de Cabeçalhos

O *overhead* de cabeçalho representa o "desperdício" de banda gerado pelas informações de controle em relação aos dados úteis (payload) que estão sendo transmitidos.

Como o QAP possui um cabeçalho fixo extremamente enxuto de apenas **4 bytes**, seu desempenho é altamente eficiente. A porcentagem de overhead varia de acordo com o tamanho da pergunta ou resposta:

Cenário A: Payload muito pequeno (Ex: "quanto é 2+2?")

★ **Payload:** 13 bytes.

- ★ **Cabeçalho:** 4 bytes.
- ★ **Tamanho total do pacote de aplicação:** 17 bytes.
- ★ **Overhead do QAP:** ~23,5% (4 bytes de controle para cada 13 bytes de dados).

Cenário B: *Payload médio* (Ex: "qual o sentido da vida?")

- ★ **Payload:** 23 bytes.
- ★ **Cabeçalho:** 4 bytes.
- ★ **Tamanho total do pacote de aplicação:** 27 bytes.
- ★ **Overhead do QAP:** ~14,8%.

O protocolo é altamente otimizado para aplicações onde o poder de processamento e a largura de banda são limitados. Ele não gasta nenhum byte desnecessário com metadados complexos.

Parte 4: Comparação com Protocolos Padrão

Para contextualizar a eficiência do QAP, é fundamental compará-lo com protocolos amplamente adotados na indústria, especialmente considerando cenários de sistemas embarcados e redes restritas.

QAP vs. HTTP (Hypertext Transfer Protocol)

O HTTP é o padrão da web, mas é notoriamente ineficiente para trocas de mensagens muito curtas e rápidas.

- ★ **Estrutura:** Baseado em texto plano.
- ★ **Overhead:** Extremamente alto. Uma simples requisição HTTP GET carrega dezenas de metadados em seu cabeçalho (Host, User-Agent, Accept, Connection, etc.). O tamanho do cabeçalho facilmente ultrapassa **200 a 500 bytes**.
- ★ **Comparativo prático:** Se a senhora enviasse a string "quanto é 2+2?" (13 bytes) via HTTP, o pacote total poderia ter cerca de 300 bytes. O overhead seria superior a **95%**. O QAP vence o HTTP com facilidade no quesito economia de tráfego de rede e tempo de processamento.

QAP vs. MQTT (Message Queuing Telemetry Transport)

O MQTT é o padrão ouro para aplicações de Internet das Coisas (IoT), leitura de sensores e comunicação entre microcontroladores.

- ★ **Estrutura:** Binário, assim como o QAP.
- ★ **Overhead:** O MQTT foi projetado para ser o mais leve possível, possuindo um cabeçalho

fixo de apenas **2 bytes** (sendo que o campo de tamanho de payload pode escalar em bytes adicionais caso a mensagem seja muito grande).

★ **Comparativo prático:** Em termos estritos de bytes trafegados, o MQTT (2 bytes) e o QAP (4 bytes) estão quase empatados em eficiência bruta. Ambos são excelentes para redes restritas.

★ **Diferenças Importantes:** A grande vantagem do MQTT sobre o QAP não está no overhead, mas na sua arquitetura *Publish/Subscribe* (que permite enviar dados para múltiplos clientes simultaneamente) e nos recursos de QoS (Quality of Service), que garantem a entrega da mensagem mesmo em conexões instáveis de sensores remotos. O QAP, por outro lado, é mais fácil de implementar do zero em código C/Python nativo, pois não requer um *Broker* intermediário operando a rede.

Conclusão

Ao implementar o protocolo QAP sobre *sockets* puros, nós conseguimos atingir uma eficiência de cabeçalho de apenas 4 bytes, um valor comparável a protocolos de nicho de alta performance voltados para redes restritas, como o MQTT (2+ bytes). Essa arquitetura provou ser vastamente superior ao padrão HTTP (que facilmente ultrapassa os 100 bytes de *overhead*) para cenários estritos de troca de mensagens curtas. Com essa abordagem minimalista em nível de aplicação, o protocolo elimina a pesada carga de processamento de *parsing* (análise léxica) exigida por mensagens baseadas em texto e economiza significativamente a largura de banda, consolidando-se como uma solução altamente otimizada para a rede do projeto.

Referências

FOROUZAN, B. A., MOSHARRAF, F. Redes de Computadores - Uma Abordagem Top-Down. São Paulo: McGraw Hill, 2013. ISBN: 9788580551686.

TANENBAUM, A. S., WETHERALL, D. Redes de Computadores. 6ª Edição. São Paulo: Pearson, 2021. ISBN: 9788582605615.

OASIS. **MQTT Version 5.0. OASIS Standard**. Editado por Andrew Banks e Rahul Gupta. 2019. Disponível em: <[MQTT Version 5.0 | OASIS Standard](#)>.

FIELDING, R.; RESCHKE, J. **Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing**. RFC 7230, Internet Engineering Task Force (IETF), 2014. Disponível em: <[RFC 7230 - Hypertext Transfer Protocol \(HTTP/1.1\): Message Syntax and Routing](#)>.